

LABORATORY EXERCISE 10

Cloud Computing and AI Model Deployment for Engineering Applications

Aim:	Deploy a trained AI model as an interactive web application using Streamlit and manage workflow via Git/GitHub.
Required Tools:	Python 3.x, Streamlit, TensorFlow 2.19+, NumPy, Pandas, Scikit-Learn, Joblib, Git & GitHub.
GitHub Username:	[Your-GitHub-Username]
Project Repository:	https://github.com/username/ai-model-deployment
Streamlit App URL:	https://ai-model-deployment.streamlit.app

Observations, Results & Discussion Answers

1. Import All Necessary Libraries and Tools

a. Which libraries are required to load saved models, preprocess data, create a Streamlit app, and visualise results?

Streamlit creates the web UI, TensorFlow/Keras loads models, NumPy/Pandas manage datasets, and Joblib/JSON load tokenizers.

b. Write the import statements for these libraries.

```
import streamlit as st; import tensorflow as tf; import numpy as np; import pandas as pd;
import json; import joblib
```

2. Revisit Lab 8 to Save Models and Tokenisers

a. What models and tokenisers were trained and saved? List their names.

The saved artifacts are the trained Keras classification model (`best_model.h5`) and the tokenizer configuration (`tokenizer.json`).

b. Write code to save the BEST model and any tokeniser using Keras and Python JSON module.

```
model.save('best_model.h5') and with open('tokenizer.json', 'w') as f:
    json.dump(tokenizer.to_json(), f)
```

3. Create a Streamlit Application Structure

a. What are the key components of a basic Streamlit app?

The key components are the user header/title UI elements, input interactive widgets, feature pre-processing, inference logic, and result display components.

b. Write sample code to structure the app with a title, input section, and output section.

```
st.title("AI Model Deployment"); user_input = st.text_input("Enter text:");  
st.write(f"Result: {user_input}")
```

4. Load Saved Models and Tokenisers

a. Write code to load the saved model and tokenisers.

```
model = tf.keras.models.load_model('best_model.h5') and with open('tokenizer.json') as f:  
tokenizer = tf.keras.preprocessing.text.tokenizer_from_json(json.load(f))
```

b. How do you ensure loaded objects match feature names and types used during training?

Preprocess dynamic inputs to strictly match the shape, datatype, order, and padding lengths expected by `model.input_shape`.

5. Setup and Collect User Inputs

a. Identify all fields required for user input based on the task.

Fields depend on data input type: text area for NLP prompts, or sliders/numeric input fields for tabular engineering features.

b. Write Streamlit code to create input widgets for each field.

```
input_text = st.text_area("Input Text Data:") or num_val = st.number_input("Enter Numeric  
Value:", min_value=0.0)
```

c. Write code to collect inputs for prediction.

```
user_inputs = {'text': input_text, 'feature_val': num_val} aggregates widget values dynamically  
upon user interaction.
```

6 & 7. Prediction Trigger & Tokenizer Application

6a & 6b. Add a button to trigger prediction and handle the click event.

`if st.button("Predict"): prediction = model.predict(processed_input)` executes model inference when triggered.

7. Apply tokenizer where necessary using saved tokenizer.

`seq = tokenizer.texts_to_sequences([input_text]); padded = tf.keras.preprocessing.sequence.pad_sequences(seq, maxlen=100)`

8. Display Prediction Results

a. Write code to display the prediction result in an informative way.

`st.success(f"Predicted Class: {class_names[np.argmax(prediction)]}")` displays class output prominently on UI.

b. Write code to include a confidence score and message based on threshold.

`conf = np.max(prediction); st.info(f"Confidence: {conf:.2%}")` if `conf > 0.5` else `st.warning("Low Confidence Prediction")`

9. Set Up Streamlit and Deploy App

a. How do you run the Streamlit app locally?

Execute `streamlit run app.py` in the terminal after saving the code script to `app.py`.

b & c & d. Deployment process on Streamlit Community Cloud.

Create a `requirements.txt` file, push code and model artifacts to GitHub, connect repo to Streamlit Cloud, and deploy.

e. Verification.

Access the deployed `.streamlit.app` URL in web browser to test real-time input inference and stability.

10. Summary Report & Challenges

Deployment Summary & Reflection Report (100-150 words):

This project successfully deployed an artificial intelligence classification model trained in Laboratory Exercises 8 and 9 into a cloud-hosted web application using Streamlit. Users input engineering parameter data or text prompts into the streamlined web UI, which is then dynamically tokenized and passed into the saved Keras TensorFlow model for live inference. Key challenges included dependencies mismatch during cloud container startup, which was resolved by freezing precise library versions in `requirements.txt`. Additionally, shape dimension mismatches between input widgets and tokenizer outputs were resolved by applying strict sequence padding (`maxlen=100`). The continuous integration pipeline via GitHub and Streamlit Cloud ensures automated updates and robust engineering deployment.